English recording script for a twelve minute presentation

FTEC5660 Individual Project 2: Reproducibility

SANG Yuchen | 1155262204

The target duration is approximately 12 minutes, including screen changes, and the assignment allows a maximum of 15 minutes. The spoken text contains approximately 1,590 words, with a suggested pace of about 145 words per minute and approximately one minute for screen changes. Read the paragraphs marked Say while using the Screen paragraphs as presentation directions, then rehearse with a timer and adjust the length of examples to follow the schedule below.

Prepare before recording

1. Open FTEC5660_Presentation.pdf at page 1 and use this document as the main slide presentation throughout the recording.

2. Open FTEC5660_LiTS_DeepSeek_Report.pdf and prepare quick access to pages 1, 3 and 4 for the discussion of scope, results and evaluation.

3. Open the public submission repository, its commit history, assignment/experiment_plan.json, assignment/study.py, lits/lm/openai_chat.py, assignment/DEBUG_DIARY.md, and assignment/evidence/steps1_seed11/ in browser tabs.

4. Open assignment/deliverables/formal_trials.csv and assignment/deliverables/format_audit.json, using the browser code view to display the saved records clearly.

5. Prepare .\.venv\Scripts\python.exe -m assignment.study preflight in a terminal at the repository root, run it once before recording, and keep the successful output available for the demonstration. This command performs an offline check with zero model calls.

6. Close .env, chats containing keys and unrelated tabs, then prepare Zoom screen sharing and recording with the presentation windows visible and this script outside the shared area where possible.

Screen plan

Time	Main slide	Open while speaking	Show / point to
00:00-00:45	1	Slides only	Name, student ID, research question
00:45-02:00	2	Report page 1; then return to slide 2	Table 6 reference; workflow and comparison conditions
02:00-03:15	3	assignment/experiment_plan.json	Source revision, dataset selection, model and endpoint
03:15-04:30	4	Plan study section; assignment/study.py	Depth limits 1/3/10; seeds; enforced depth limit
04:30-05:40	5	formal_trials.csv; report page 3	Nine trials; primary mean and sample SD
05:40-06:40	6	Plot on slide 6	Token use and timing conditions
06:40-07:45	7	Report page 3 observed depth table	Deepest node 4 with maximum depth 10
07:45-09:00	8	lits/lm/openai_chat.py; assignment/DEBUG_DIARY.md	DeepSeek adapter; Windows failures and retained attempts
09:00-10:40	9	Evidence folder; format_audit.json	Manifest, request records, tree data and formatting examples
10:40-12:00	10	GitHub README and commits; terminal preflight	Reproduction steps and successful offline verification

00:00-00:45: Slide 1, Introduction

Screen: Start Zoom recording with slide 1 visible and point to the research question while introducing the project.

Say:

Hello, my name is SANG Yuchen, my student ID is 1155262204, and this presentation introduces my reproducibility project for FTEC5660. I selected LiTS, a modular framework for language model tree search, and ran the authors' mathematical reasoning workflow through the DeepSeek API. My experiment examines how maximum search depth affects accuracy and resource use by comparing three depth limits on the same thirty questions, with three repeated runs for each setting. During this presentation, I will explain the experimental setup, discuss the measured results, and show the saved evidence and instructions that support reproduction of the workflow.

00:45-02:00: Slide 2, Reproduction target and scope

Screen: Show slide 2, switch to the Objective and scope section on report page 1, highlight the paragraph about Table 6, and return to slide 2.

Say:

LiTS qualifies as an agentic system because it searches over alternative reasoning trajectories across multiple steps, using a policy to propose candidate actions, a transition component to extend each trajectory, and a reward component to evaluate those candidates. Monte Carlo Tree Search determines which branches to explore, after which the saved terminal answers pass through the repository's voting and numeric checking workflow.

My reference is Table six of the paper, where ReST with MCTS reports thirty seven percent accuracy on one hundred numerical answer MATH500 questions. My experiment uses DeepSeek, an API based generative reward component, and a smaller subset, so interpreting the comparison requires attention to these differences in evaluation conditions. The reproduction target covers the runnable workflow and its evaluation behavior under those documented substitutions, while the controlled comparison examines depth settings within a shared DeepSeek setup. This design supports conclusions about the tested depth settings under the recorded conditions, with the original paper providing context for the selected method and benchmark.

02:00-03:15: Slide 3, Environment, data and model

Screen: Show slide 3, open assignment/experiment_plan.json, and point to upstream_commit, dataset and model while leaving the long hashes visible for reference.

Say:

The saved experiment plan records a specific Git commit for the upstream source and a specific Hugging Face revision for the dataset, giving future readers identifiable versions of both inputs. The source file contains three hundred and sixteen numerical answer rows, and my evaluation uses the first thirty in their original order, with the dataset checksum verified during loading.

The environment uses an isolated Python three point twelve installation on Windows with the dependencies required by the API mathematics workflow. The recorded model name is deepseek-flash, served through the DeepSeek endpoint with internal thinking disabled, and the experiment records actual request parameters and token usage because different roles use different decoding settings. Local seeds control local randomness, while the hosted provider introduces additional variation in model responses. Fixing the source and data versions therefore supports reproducibility of the local setup, alongside a continuing dependence on the provider's model service and its future behavior.

03:15-04:30: Slide 4, Controlled modification

Screen: Show slide 4 and the plan's study section, then open assignment/study.py and search for force_terminating_on_depth_limit=True.

Say:

The main experimental factor is maximum search depth, with limits of one, three and ten and a baseline setting of ten. Every setting uses the same thirty questions and three local seeds, eleven, twenty two and thirty three, producing nine complete runs and two hundred and seventy evaluations of thirty distinct questions.

The remaining search settings stay fixed at ten MCTS iterations, a branching factor of three and a rollout length of two, while forced termination at the depth limit is enabled for every group. This option makes the configured maximum an enforced limit on the search trajectory and gives the depth comparison a consistent implementation across settings.

A separate pilot using five questions informed the experimental grid before formal execution because its trajectories frequently stopped at depth one. I changed the initial grid of five, ten and fifteen to one, three and ten to include a setting that constrained the observed depth, and I disclosed the overlap between the pilot questions and the formal exploratory subset.

04:30-05:40: Slide 5, Accuracy and repeated runs

Screen: Show slide 5, open formal_trials.csv to display all nine rows, and switch to the aggregate table on report page 3.

Say:

The CSV contains measured results from every completed depth and seed combination, with accuracy calculated over all thirty planned questions in each run and missing final answers counted as incorrect. At depth one, mean accuracy is ninety percent with zero sample standard deviation across three runs, while depth three also achieves ninety percent with a standard deviation of three point three percentage points. At depth ten, mean accuracy is eighty five point six percent with a standard deviation of five point one percentage points.

These deviations describe variation between repetitions on the same small dataset, so conclusions about the wider MATH500 population require additional evaluation. The matching depth one scores establish consistency in aggregate accuracy across those three repetitions, while response level consistency would require a separate comparison of generated outputs. Across the tested settings, the measured accuracy provides limited evidence of improvement from increasing the maximum depth, with the resource measurements providing additional information for comparing the settings.

05:40-06:40: Slide 6, Resource measurements

Screen: Keep slide 6 visible and point to accuracy, tokens and time in that order while explaining the resource measurements.

Say:

The plot shows average token use of approximately eight thousand four hundred and sixty one per question at depth one, ten thousand nine hundred and eighty four at depth three, and eleven thousand three hundred and seventy three at depth ten. These totals include search and final answer voting, with depth one using roughly twenty six percent fewer tokens than depth ten across the recorded runs.

Average recorded time is approximately fourteen point six, sixteen point nine and seventeen point five seconds per question, although execution conditions affect the interpretation of these measurements. The initial batch ran nine processes, and a replacement run executed alone after a Windows file writing failure, creating different load conditions across the timing records. I therefore treat time as descriptive and use token counts as a measure of resource consumption, with monetary cost requiring a separate calculation using the applicable

provider prices.

06:40-07:45: Slide 7, Meaning of a search step

Screen: Show slide 7, switch to the observed depth table on report page 3, point to the deepest node of four under the configured maximum of ten, and return to slide 7.

Say:

One external search action can contain a complete derivation with several mathematical operations, which helps explain the accuracy achieved by the depth one setting. Maximum search depth controls the number of external actions along a trajectory, while arithmetic operations and model reasoning can occur within an individual generated action.

The observed depth table shows that the deepest saved node in the depth ten group reaches four, with many trajectories ending earlier through the system's termination behavior. Model calls can exceed tree depth because the workflow also samples alternatives, scores candidates and performs voting, making actual depth and call count useful complementary measurements.

The configured limit also determines when LiTS supplies its final step instruction, so the intervention includes this prompt behavior alongside the permitted trajectory length. Additional search therefore creates opportunities for further evaluation and higher resource use, while the resulting answer quality remains an empirical question for each task and configuration.

07:45-09:00: Slide 8, Compatibility repairs and execution failures

Screen: Show slide 8, search for deepseek and strict in lits/lm/openai_chat.py, and open the UTF-8 and completion marker entries in assignment/DEBUG_DIARY.md.

Say:

Running the workflow required a compatible Python environment, removal of an unused torch import from the API configuration, and a DeepSeek specific mapping for the model wrapper's thinking option. The stable endpoint also required omission of a beta only strict tool flag while retaining the schema and local validation, and every formal group used the same compatibility repairs.

An early test on Windows completed its API requests before failing while saving a Unicode character, which led to the launcher using UTF-eight for artifact writing. A later depth ten, seed eleven attempt failed during replacement of an atomic completion marker, after which I retained the incomplete attempt and repeated the entire configuration once with the same experimental source. The retry followed the execution failure and preserved the planned configuration and evaluation procedure.

All nine selected formal runs have matching recorded source hashes, and a later monitoring repair avoids opening active completion files. The diary records concurrent file access as a possible contributor to the replacement failure, with its precise cause remaining uncertain.

09:00-10:40: Slide 9, Evidence, scoring and limitations

Screen: Show slide 9, open assignment/evidence/steps1_seed11/, and display run_manifest.json, api_calls.jsonl and the beginning of full_trees.jsonl before showing one formatting example in assignment/deliverables/format_audit.json.

Say:

The evidence folder supports inspection of the configuration, run manifest, request records, evaluation output, full search trees and terminal nodes, providing the records used to calculate the final table. The manifest records run status and settings, the request records describe actual model calls, and the evidence index identifies the selected runs and excluded attempts. A restoration script reconstructs the raw run directories from these saved

artifacts using an offline procedure.

The scoring audit identified an evaluation limitation because the original numeric checker can reject answers containing units or currency, including seventy two degrees against seventy two and twelve centimetres against twelve. The audit identified sixteen formatting related false negatives across three known tasks in the formal evaluations, and the primary results continue to use the original checker scores. This diagnostic helps explain how answer formatting contributes to the reported accuracy and provides a specific issue for future evaluator validation.

The study also uses an ordered subset of thirty distinct questions with pilot overlap, an evolving remote model alias, and model based reward scoring and answer grouping. These conditions limit generalization and allow formatting or answer selection behavior to contribute to small score differences. A larger held out sample and a separately validated normalization rule established before execution would improve a future evaluation of the depth settings.

10:40-12:00: Slide 10, Reproduction demonstration and conclusions

Screen: Show slide 10, open the public repository README and commit history, and point to the installation instructions and report link before running .\.venv\Scripts\python.exe -m assignment.study preflight in the prepared terminal. Return to slide 10 for the closing sentences, using the saved successful output as a clearly identified prior check if the live command encounters a problem.

Say:

The public repository provides the report, slides, experimental evidence, dependency lock and reproduction instructions, with separate commits supporting review of the adapter, experiment setup, repairs and results. API keys remain in the ignored local environment file, and the offline preflight command verifies the prepared configuration and data with zero model calls. Repeating the paid experiment requires a personal DeepSeek key and the documented command for each depth and seed, while saved evidence and report generation remain available through the offline procedures.

The experiment shows that the API based LiTS workflow runs under the documented conditions, with the default depth of ten achieving eighty five point six percent accuracy on average and depth one achieving ninety percent with lower token use on this subset. I recommend measuring actual depth and resource consumption before increasing the search limit, alongside evaluating answer normalization before attributing small differences to reasoning quality. Thank you for your attention to this project and its findings.

After recording

Stop recording and allow Zoom to finish processing, then review the video for audio quality, screen readability and duration before sharing it through an accessible link. Update VIDEO_LINK.md with the recording link and duration, commit the change, and verify that the teacher can access the recording through the chosen sharing settings. Submit the report PDF, public GitHub URL and recording link through the required course channel before the stated deadline, with the completed video providing the presentation component of the submission.